Computer Networks Lab

Short Exam Programs & Procedure

1. CRC Sender Side (CRC-CCITT)

Aim: Generate CRC remainder at the sender side.

```
#include <stdio.h>
#include <string.h>

int main() {
    char data[100], temp[100];
    char poly[] = "10001000000100001";
    int n, m = strlen(poly);

    printf("Enter data: ");
    scanf("%s", data);

    n = strlen(data);
    strcpy(temp, data);
    for(int i=0; i<m-1; i++) temp[n+i]='0';
    temp[n+m-1]='\0';

    for(int i=0; i<n; i++)
        if(temp[i]=='1')
            for(int j=0; j<m; j++)
                temp[i+j] = (temp[i+j]==poly[j]) ? '0':'1';

    printf("CRC: ");
    for(int i=n; i<n+m-1; i++) printf("%c", temp[i]);

    return 0;
}
```

Note: This short program demonstrates CRC-CCITT only. Use the corresponding generator polynomial for CRC-12 and CRC-16.

2. CRC Receiver Side (CRC-CCITT)

Aim: Check the received codeword for transmission errors.

```
#include <stdio.h>
#include <string.h>

int main() {
    char data[100];
    char poly[] = "10001000000100001";
    int n, m = strlen(poly), error = 0;

    printf("Enter received codeword: ");
    scanf("%s", data);

    n = strlen(data);

    for(int i=0; i<=n-m; i++)
        if(data[i]=='1')
            for(int j=0; j<m; j++)
                data[i+j] = (data[i+j]==poly[j]) ? '0':'1';

    for(int i=n-m+1; i<n; i++)
        if(data[i]=='1') error=1;

    if(error) printf("Error detected");
    else printf("No error");

    return 0;
}
```

3. Distance Vector Routing

Aim: Calculate shortest distances between network nodes.

```
#include <stdio.h>
#define INF 999

int main() {
    int n, c[10][10], d[10][10];
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter cost matrix (999 = no link):\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++) {
            scanf("%d", &c[i][j]);
            d[i][j] = c[i][j];
        }

    for(int k=0; k<n; k++)
        for(int i=0; i<n; i++)
            for(int j=0; j<n; j++)
                if(d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];

    for(int i=0; i<n; i++) {
        printf("\\nRouting table of Node %d\\n", i);
        for(int j=0; j<n; j++)
            printf("%d -> %d = %d\\n", i, j, d[i][j]);
    }

    return 0;
}
```

Note: This compact version displays shortest distances. A full routing table also includes the next-hop node.

4. Packet Capture Using Wireshark

Aim: Capture and analyze network packets using Wireshark.

Procedure:

1. Open Wireshark.
2. Select the active Wi-Fi or Ethernet interface.
3. Click Start Capture.
4. Open a website to generate network traffic.
5. Observe the captured packets.
6. Apply a display filter and stop the capture.

Useful display filters:

```
tcp\nudp\nhttp\ndns\nicmp
```

Result: Network packets were captured and analyzed using Wireshark.

Compile and run C programs:

```
gcc filename.c -o output\n./output        (Linux/macOS)\noutput.exe      (Windows)
```